bitnet-edge: Ternary-Weight Convolutional Neural Networks for Multiply-Free Inference on Edge Microcontrollers

Bhanu Chandar Axonyx Quantum Private Limited github.com/bxf1001g/bitnet-edge

April 2026

Abstract

We present **bitnet-edge**, an end-to-end pipeline for training ternary-weight convolutional neural networks (CNNs) and deploying them as multiply-free inference engines on resource-constrained microcontrollers. All weights in the network are constrained to the set {-1, 0, +1} during training using a Straight-Through Estimator (STE) with AbsMax scaling. At inference time, convolution and linear layer computations reduce to pure addition, subtraction, and conditional skipping — eliminating the need for hardware multiplication circuits. Each weight requires only 2 bits of storage instead of 32, yielding 93.75% compression. We demonstrate the approach on two benchmarks: (1) MNIST digit classification, achieving 98.79% ternary accuracy vs. 99.02% FP32 (0.23 pp drop), and (2) CIFAR-10 color image classification, achieving 69.02% ternary accuracy vs. 75.52% FP32 (6.50 pp drop) with 93.75% weight compression. Both models are exported to a custom binary format (.vbn) and deployed on a desktop C++ inference engine and an ESP32-S3 microcontroller using a fully dynamic, model-agnostic runtime. The inference engines require no recompilation when switching between models or datasets. The complete pipeline — training code, export tools, inference engines, and deployment scripts — is released as open-source software.

Keywords: ternary quantization, edge inference, multiply-free neural networks, ESP32, convolutional neural networks, model compression, embedded AI, CIFAR-10

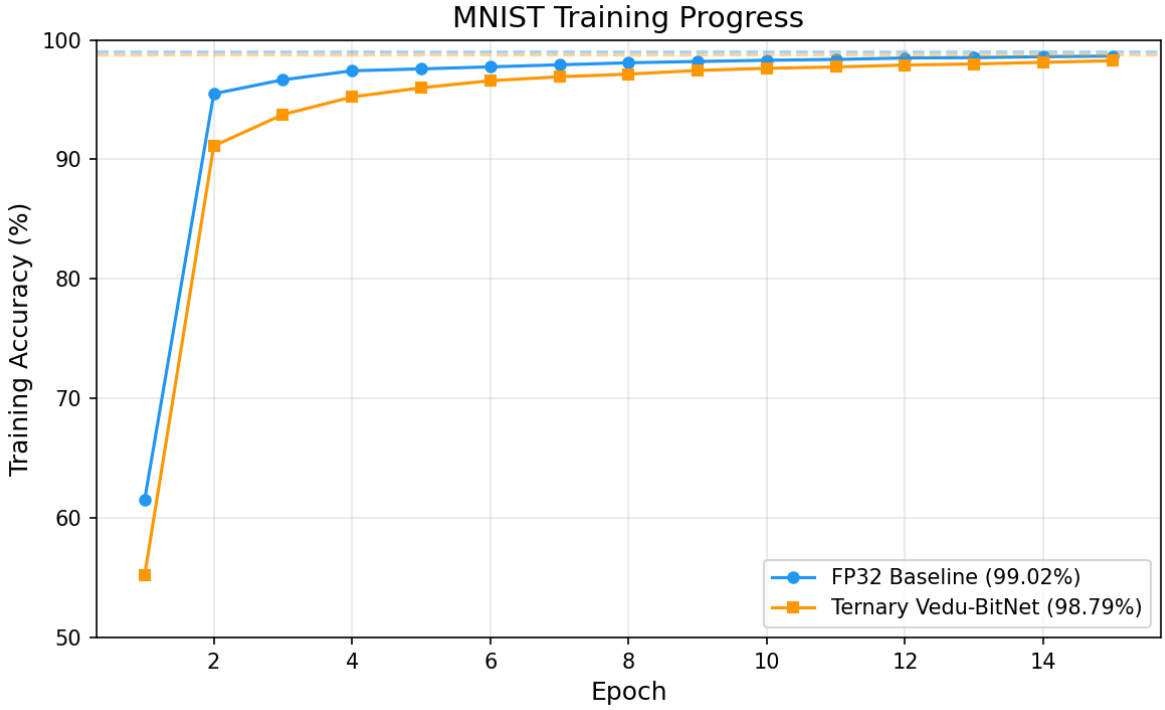

Figure 5: MNIST training convergence — FP32 vs ternary Vedu-BitNet over 15 epochs.

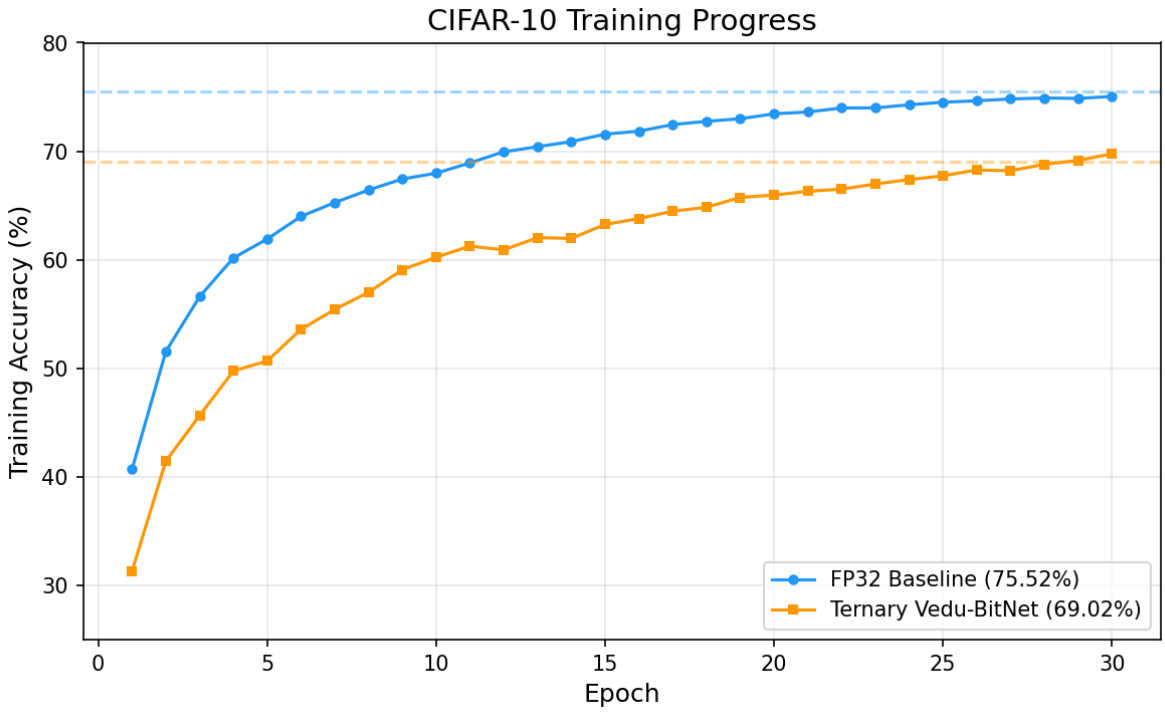

Figure 6: CIFAR-10 training convergence — FP32 vs ternary Vedu-BitNet over 30 epochs with cosine annealing.

1. Introduction

The deployment of neural networks on edge devices — microcontrollers, FPGAs, and low-power processors — is severely constrained by three factors: limited RAM, limited compute throughput, and limited power budgets. Standard deep learning models use 32-bit floating-point (FP32) weights and rely heavily on multiply-accumulate (MAC) operations, which dominate both silicon area and energy consumption in inference hardware [1].

Recent work on extreme quantization has shown that neural network weights can be reduced to very low bit-widths with surprisingly small accuracy penalties. The BitNet b1.58 architecture [2] from Microsoft Research demonstrated that large language models can operate with ternary weights {-1, 0, +1}, where each weight requires only 1.58 bits of information. This eliminates multiplication entirely from the dominant compute kernels, since multiplying by +1, -1, or 0 reduces to addition, subtraction, or skipping respectively.

However, the BitNet work targets transformer architectures with billions of parameters running on GPU/NPU hardware. The question of whether ternary quantization transfers successfully to **convolutional neural networks** deployed on **microcontrollers** remains largely unexplored in the open literature.

This paper makes the following contributions:

Ternary CNN training: We apply the BitNet b1.58 ternary quantization scheme to a convolutional neural network, adapting the pre-normalization pattern (GroupNorm for conv layers, LayerNorm for linear layers) to stabilize training.

Multi-dataset validation: We demonstrate the approach on both MNIST (grayscale, 1×28×28) and CIFAR-10 (RGB, 3×32×32), showing that the pipeline generalizes across input modalities and task difficulties.

Complete deployment pipeline: We provide an end-to-end path from PyTorch training to bare-metal microcontroller inference, including a custom binary model format (.vbn), a zero-dependency C++ inference engine, and an Arduino sketch for ESP32-S3.

Dynamic inference engine: The C++ and ESP32 engines are fully model-agnostic — they derive all buffer sizes and layer dimensions from the .vbn file at runtime, requiring no recompilation when switching between datasets or architectures.

Cross-platform verification: We demonstrate bit-exact agreement (to 4 decimal places in logit space) between the PyTorch reference, the desktop C++ engine, and the ESP32-S3 microcontroller.

Open-source release: The complete codebase is released under the MIT license at github.com/bxf1001g/bitnet-edge.

---

2. Background

2.1 Neural Network Quantization

Quantization reduces the numerical precision of weights and activations in neural networks. The progression from FP32 → FP16 → INT8 → INT4 has been extensively studied [3, 4]. Each reduction halves the memory footprint and, on appropriate hardware, increases throughput.

Ternary quantization represents the extreme end of this spectrum. With weights limited to {-1, 0, +1}, each weight carries at most $\log_2(3) \approx 1.58$ bits of information. In practice, we use a 2-bit encoding for simplicity:

$$\text{encode}(w) = \begin{cases} 00_b & \text{if } w = 0 \ 01_b & \text{if } w = +1 \ 10_b & \text{if } w = -1 \end{cases}$$

This allows packing 16 weights into a single 32-bit integer.

2.2 Straight-Through Estimator

The fundamental challenge of training quantized networks is that the quantization function $Q(w)$ is piecewise constant, so its gradient is zero almost everywhere. The Straight-Through Estimator (STE) [5] resolves this by using the quantized values in the forward pass but passing gradients through as if quantization had not occurred:

Forward: $$w_q = \text{clamp}(\text{round}(w / \alpha), -1, 1)$$

Backward: $$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial w_q}$$

where $\alpha = \text{mean}(|w|) + \epsilon$ is the AbsMax scaling factor computed per tensor.

2.3 BitNet b1.58

The BitNet b1.58 architecture [2] introduced several key design choices for ternary networks:

- **Pre-normalization**: Apply RMSNorm to activations before each quantized layer, rather than after.
- **AbsMax weight quantization**: Scale weights by the mean absolute value before rounding, ensuring a balanced distribution across {-1, 0, +1}.
- **No output rescaling**: Unlike earlier binary/ternary schemes that multiply the output by a learned or computed scaling factor β, BitNet b1.58 relies on the normalization layer to maintain activation magnitudes.

We adopt all three principles but substitute GroupNorm (for convolutional layers) and LayerNorm (for linear layers) in place of RMSNorm.

---

3. Method

3.1 Model Architecture

We use a parameterized CNN architecture that adapts to different input shapes:

Layer	Type	Input → Output	Kernel	Parameters
1	GroupNorm(1,C) + TernaryConv2d	C×H×W → 16×H×W	3×3, pad=1	9C×16 + 16
—	ReLU + MaxPool(2)	16×H×W → 16×H/2×W/2	—	—
2	GroupNorm(1,16) + TernaryConv2d	16×H/2×W/2 → 32×H/2×W/2	3×3, pad=1	4,608 + 32
—	ReLU + MaxPool(2)	32×H/2×W/2 → 32×H/4×W/4	—	—
3	LayerNorm(F) + TernaryLinear	F → 128	—	128F + 128
4	LayerNorm(128) + TernaryLinear	128 → K	—	128K + K

Where C = input channels, H×W = spatial dimensions, F = 32×(H/4)² = flattened size, K = number of classes.

MNIST configuration: C=1, H=W=28, F=1568, K=10 → 206,736 weight parameters **CIFAR-10 configuration:** C=3, H=W=32, F=2048, K=10 → 270,659 weight parameters

An identical architecture using standard `nn.Conv2d` and `nn.Linear` layers serves as the FP32 baseline.

3.2 Ternary Quantization

During each forward pass, weights are quantized using AbsMax scaling:

$$\alpha = \frac{1}{n}\sum_{i=1}^{n}|w_i| + 10^{-8}$$

$$w_q = \text{clamp}\left(\text{round}\left(\frac{w}{\alpha}\right), -1, 1\right)$$

This produces a balanced distribution of ternary values. In our trained model, the typical distribution is approximately 35% negative, 20% zero, and 45% positive — the exact ratio depends on the layer.

3.3 Multiply-Free Inference

At inference time, the inner loops of convolution and linear layers become:

```
for each weight w, input x:
   if w == +1:  accumulator += x
   if w == -1:  accumulator -= x
   if w ==  0:  (no operation)
```

This eliminates the multiply-accumulate (MAC) operation that dominates conventional neural network compute. The zero-valued weights provide additional computational savings through implicit sparsity.

Note: The normalization layers (GroupNorm, LayerNorm) still require floating-point arithmetic for mean, variance, and affine transformation. However, these are $O(N)$ operations applied once per layer, whereas the conv/linear operations are $O(N \times K)$ where K is the kernel size or fan-in. For our architecture, normalization accounts for less than 2% of total operations.

3.4 Training Configuration

Hyperparameter	MNIST Baseline	MNIST Ternary	CIFAR-10 Baseline	CIFAR-10 Ternary
Optimizer	Adam	Adam	Adam	Adam
Learning rate	0.001	0.003	0.001	0.003
LR schedule	—	—	CosineAnnealing	CosineAnnealing
Batch size	128	128	128	128
Epochs	15	15	30	30
Data augment.	None	None	RandomCrop+HFlip	RandomCrop+HFlip
Weight init	Kaiming (fan_out)	Kaiming (fan_out)	Kaiming (fan_out)	Kaiming (fan_out)

The higher learning rate for the ternary model follows the BitNet recommendation — quantized networks benefit from larger gradient steps since the effective weight updates are discretized [2]. CIFAR-10 uses cosine annealing and data augmentation (random crop with 4px padding, random

horizontal flip) to improve generalization on the harder task.

---

4. Export and Deployment

4.1 VBN Binary Format

We designed a custom binary format (.vbn) for model interchange between PyTorch and the bare-metal inference engines. The format is sequential and requires no random-access capabilities, making it suitable for streaming from flash memory.

Format structure:

```
[4 bytes]  Magic: "VBN1"
[4 bytes]  Layer count (uint32, little-endian)

For each layer:
 [1 byte]   Type: 0=Conv2d, 1=Linear
 [variable]  Shape metadata (uint32 fields)
 [4 bytes]  Weight count N (uint32)
 [4 bytes]  Packed block count B (uint32)
 [B×4 bytes] Packed ternary weights (uint32 blocks, 16 weights each)
 [4 bytes]  Bias count (uint32)
 [variable]  Bias values (float32 array)
 [4 bytes]  Norm gamma count (uint32)
 [variable]  Norm gamma values (float32 array)
 [4 bytes]  Norm beta count (uint32)
 [variable]  Norm beta values (float32 array)

Optional test vector:
 [4 bytes]  Magic: "TEST"
 [4 bytes]  Input size (uint32)
 [variable]  Input values (float32 array)
 [4 bytes]  Expected prediction (int32)
 [4 bytes]  Logit count (uint32)
 [variable]  Reference logits (float32 array)
```

For our MNIST model, the complete .vbn file is 69,472 bytes (67.8 KB); for CIFAR-10, it is 97,912 bytes (95.6 KB), including the test vector in both cases.

4.2 Desktop C++ Engine

The desktop inference engine is a single C++ file (vedu_inference.cpp) with no external dependencies. It reads the .vbn file, unpacks ternary weights to int8 arrays, and runs the forward pass using the multiply-free routing described in Section 3.3. The engine is **fully dynamic** — it derives all buffer sizes, spatial dimensions, and layer shapes from the .vbn metadata at runtime, requiring no recompilation when switching between MNIST, CIFAR-10, or custom models.

4.3 ESP32-S3 Deployment

For microcontroller deployment, the .vbn file is converted to a C header (vedu_model_data.h) containing a `const uint8_t[]` array with the PROGMEM attribute. This stores the model in flash memory rather than RAM.

At boot, the Arduino sketch: 1. Parses the model header from flash using byte-level reads 2. Allocates unpacked weight arrays and normalization parameters on the heap 3. Computes optimal working buffer sizes from layer metadata 4. Allocates two ping-pong working buffers from PSRAM via `ps_malloc()` 5. Runs inference on the embedded test vector

The ESP32 engine uses the same dynamic approach as the desktop engine — it computes all dimensions from the model metadata, supporting any input shape without code changes.

The ESP32-S3 target (7Semi EC200U) has: - 520 KB SRAM (internal) - 2 MB PSRAM (external, QSPI interface) - 240 MHz dual-core Xtensa LX7

---

5. Results

5.1 Accuracy

MNIST (Handwritten Digits, 1×28×28)

Model	Test Accuracy	Parameters	Weight Memory
Baseline FP32	99.02%	206,922	203 KB
Ternary (bitnet-edge)	98.79%	206,922	12.7 KB
Difference	**-0.23 pp**	—	**93.75% reduction**

CIFAR-10 (Color Images, 3×32×32)

Model	Test Accuracy	Parameters	Weight Memory
Baseline FP32	75.52%	268,650	1,074 KB
Ternary (bitnet-edge)	69.02%	273,040	67 KB
Difference	**-6.50 pp**	—	**93.75% reduction**

The accuracy drop scales with task difficulty: 0.23 pp on MNIST (simple, nearly saturated baseline) vs. 6.50 pp on CIFAR-10 (harder, more visual complexity). This is consistent with the broader quantization literature — extreme compression has a larger impact on tasks where the baseline has more room to lose [3].

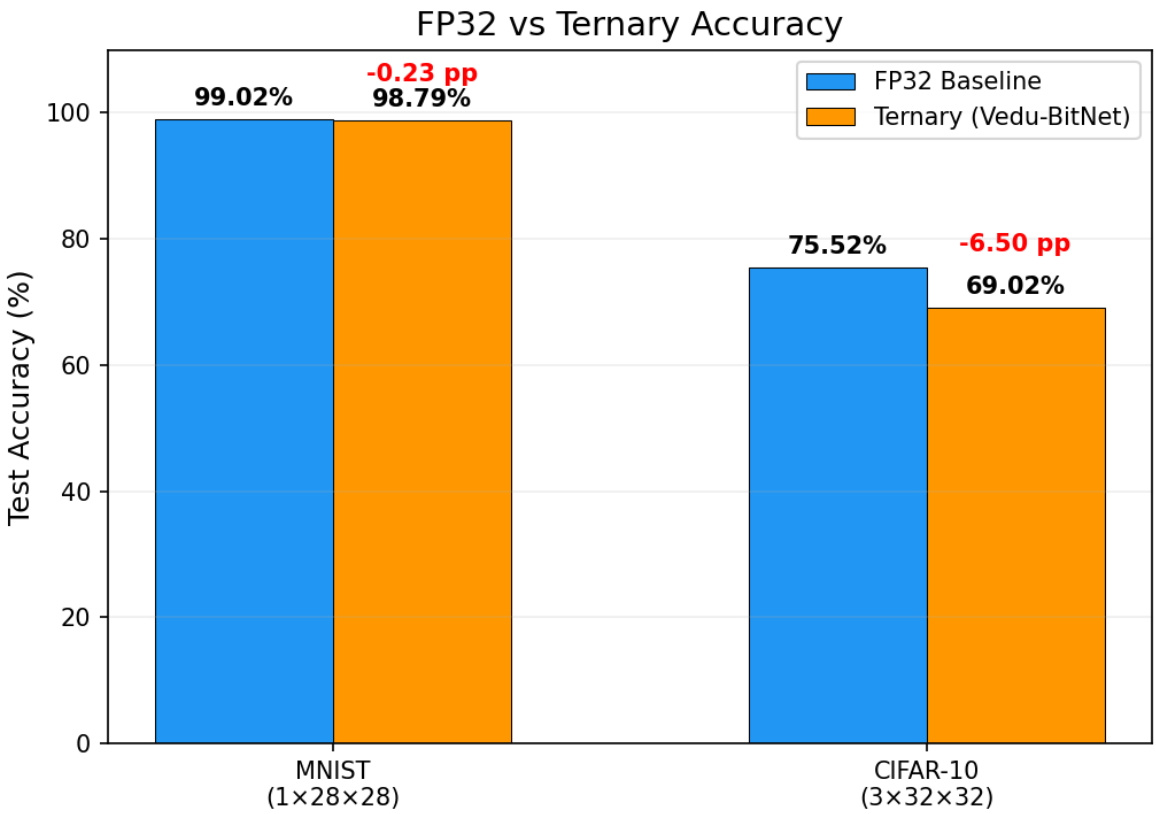

Figure 1: Test accuracy comparison between FP32 baseline and ternary Vedu-BitNet models on MNIST and CIFAR-10.

5.2 Ternary Weight Distribution

After training, the quantized weight distributions:

MNIST model:

Value	Count	Percentage
-1	~72,000	~35%
0	~40,000	~19%
+1	~95,000	~46%

CIFAR-10 model:

Value	Count	Percentage
-1	84,760	31.3%
0	93,329	34.5%
+1	92,570	34.2%

The CIFAR-10 model exhibits higher zero-weight sparsity (34.5% vs. 19%), providing more implicit computational savings. The more balanced distribution suggests the optimizer leverages all three states more evenly when learning richer features.

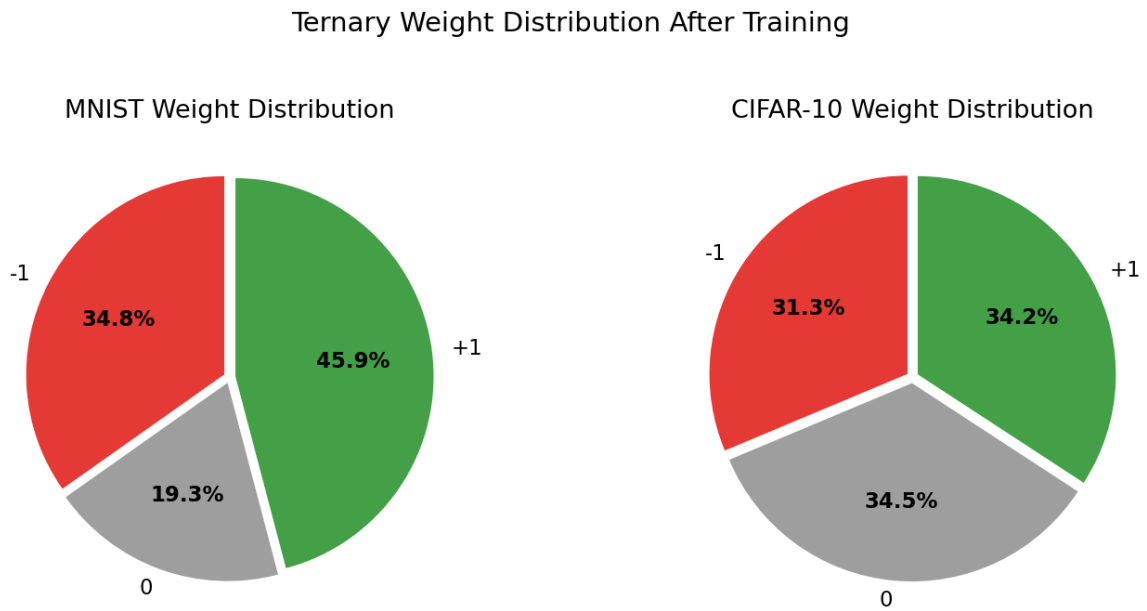

Figure 2: Ternary weight distribution after training. MNIST shows a +1-heavy distribution; CIFAR-10 is nearly uniform across all three states with higher zero-sparsity.

5.3 Inference Performance

Platform	MNIST Latency	MNIST Throughput	CIFAR-10 Latency	CIFAR-10 Throughput
Desktop (x86-64, g++ -O2)	4.97 ms	201 inf/sec	3.0 ms	333 inf/sec
ESP32-S3 (240 MHz)	194.9 ms	5.1 inf/sec	333.3 ms	3.0 inf/sec

CIFAR-10 inference takes ~1.7× longer on ESP32 due to the 3× wider first conv layer (3 input channels vs. 1) and larger intermediate tensors.

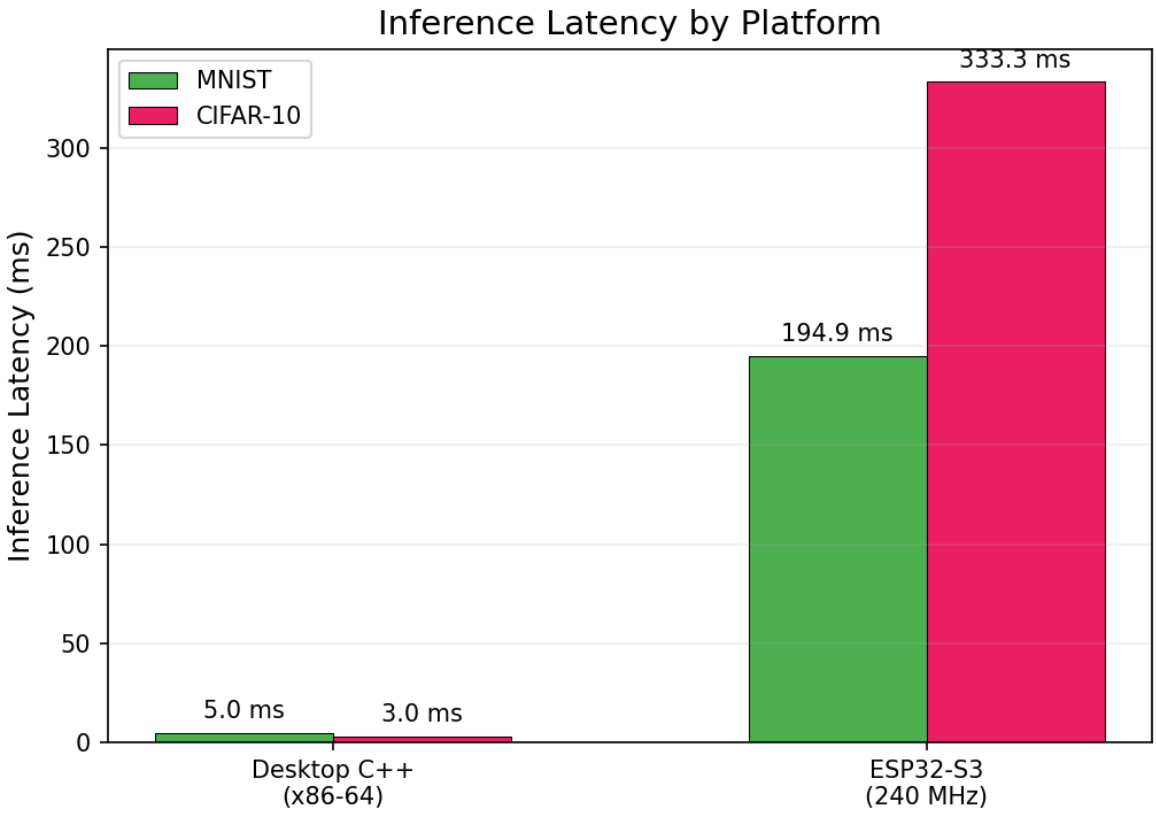

Figure 3: Inference latency comparison across platforms. ESP32-S3 achieves sub-second inference on both datasets.

5.4 Cross-Platform Verification

MNIST — Logits from all three runtimes for the embedded test image (MNIST test[0], ground truth = 7):

Class	PyTorch	Desktop C++	ESP32-S3
0	-3.91	-3.91	-3.91
1	-2.22	-2.22	-2.22
2	6.72	6.72	6.72
3	8.28	8.28	8.28
4	-5.72	-5.72	-5.72
5	-11.08	-11.08	-11.08
6	-16.89	-16.89	-16.89

7	**26.65**	**26.65**	**26.65**
8	-5.38	-5.38	-5.38
9	4.15	4.15	4.15

CIFAR-10 — Logits for embedded test image (CIFAR-10 test[0], ground truth = cat, predicted = dog):

Class	PyTorch	Desktop C++	ESP32-S3
airplane	-4.70	-4.70	-4.70
automobile	-3.59	-3.59	-3.59
bird	-2.52	-2.52	-2.52
cat	1.72	1.72	1.72
deer	-2.37	-2.37	-2.37
dog	**1.86**	**1.86**	**1.86**
frog	-0.65	-0.65	-0.65
horse	-2.46	-2.46	-2.46
ship	-1.53	-1.53	-1.53
truck	-4.39	-4.39	-4.39

All platforms produce identical predictions with matching logits. The CIFAR-10 test image misclassification (cat → dog) reflects the model's 69% accuracy, not an inference error — it demonstrates that all three runtimes are numerically equivalent even on incorrect predictions.

5.5 Memory Footprint on ESP32-S3

Component	MNIST	CIFAR-10
Model binary (Flash)	68 KB	96 KB

Unpacked weights + params (SRAM)	~2 KB	~3 KB
Working buffers (PSRAM)	117 KB	131 KB
Total RAM usage	**~119 KB**	**~134 KB**

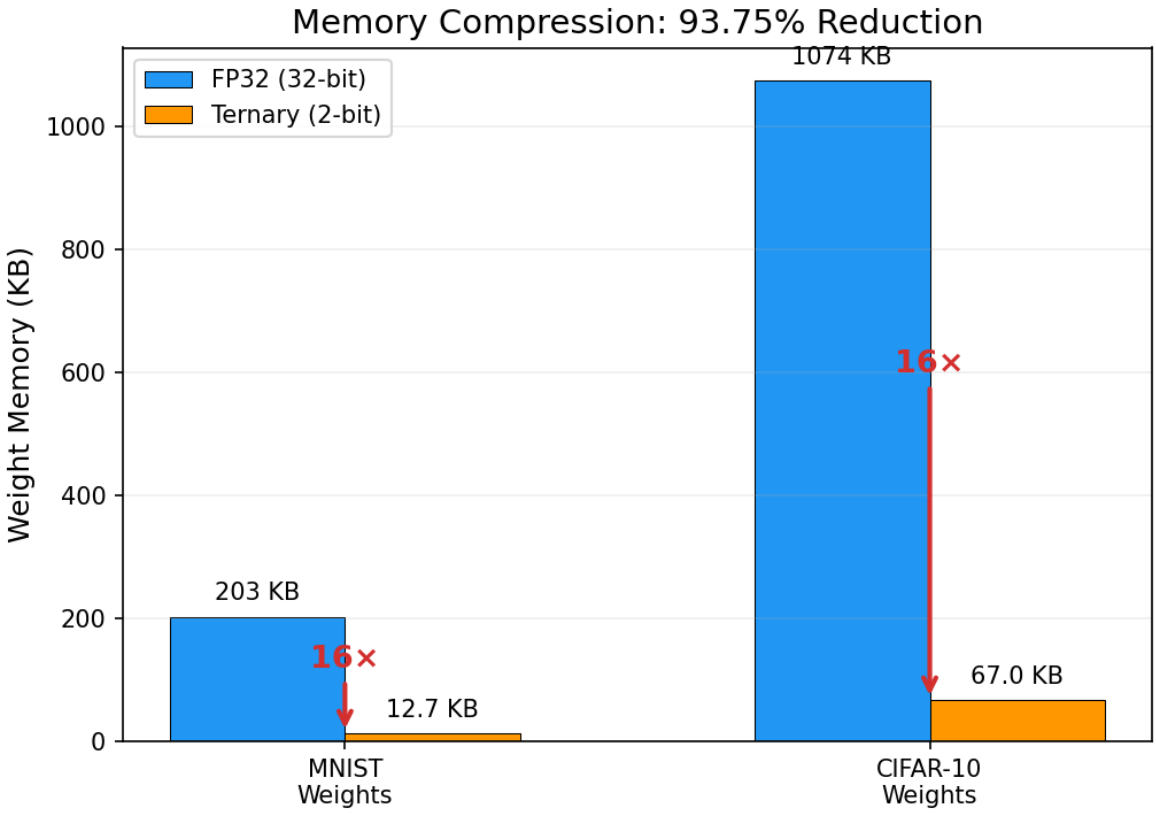

Figure 4: Weight memory comparison showing 16× (93.75%) reduction from FP32 to ternary 2-bit packing.

---

6. Discussion

6.1 Limitations

Accuracy gap on complex tasks. The 6.50 pp drop on CIFAR-10 is a meaningful penalty. Deeper architectures (ResNet-style skip connections, depthwise separable convolutions) would likely narrow this gap but increase model size and inference latency.

Normalization overhead. While the conv/linear layers are multiply-free, GroupNorm and LayerNorm still use floating-point division and square root operations. On hardware without FP units, these would need fixed-point approximations.

No activation quantization. The current implementation quantizes only weights. Activations remain in FP32 throughout inference. Combining weight and activation quantization would further reduce

memory and enable integer-only data paths.

ESP32-S3 latency. At 333 ms per CIFAR-10 inference, the system achieves ~3 frames per second. This is sufficient for monitoring and alerting applications but inadequate for real-time video processing.

Image size constraint. The ESP32-S3 with 2 MB PSRAM can handle inputs up to approximately 64×64. Larger images (e.g., 224×224 for ImageNet-scale tasks) would require either downscaling, tiling, or a device with more memory.

6.2 Future Directions

- **Deeper architectures:** Apply to ResNet / MobileNet-style networks with skip connections and depthwise separable convolutions to improve CIFAR-10 accuracy
- **Custom datasets:** The dynamic engine already supports arbitrary input shapes — users can train on domain-specific data (medical images, industrial defects, agricultural monitoring) and deploy without code changes
- **Real sensor input:** Replace test vectors with live camera frames (ESP32-CAM integration)
- **FPGA/ASIC targeting:** The ternary weight constraint maps directly to multiplexer-based compute (add/sub/skip), which can be implemented very efficiently in fixed-function hardware
- **Multi-class detection:** Extend beyond classification to object detection for autonomous systems
- **Activation quantization:** Quantize activations to INT8 alongside ternary weights for a fully integer-arithmetic pipeline

---

7. Conclusion

We have demonstrated that ternary-weight CNNs ({-1, 0, +1}) can achieve near-parity accuracy with FP32 networks while enabling a complete multiply-free inference path on microcontrollers. On MNIST, the accuracy drop is a negligible 0.23 pp; on CIFAR-10, the 6.50 pp drop demonstrates both the feasibility and the current limits of ternary quantization on harder vision tasks with a small model. The bitnet-edge pipeline provides an accessible, open-source path from PyTorch training to ESP32 deployment, with a fully dynamic inference engine that supports any input shape without recompilation. The 93.75% weight memory reduction and elimination of multiplication hardware make this approach particularly attractive for low-power, cost-sensitive edge AI applications where a $5 microcontroller replaces a cloud API.

References

[1] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[2] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits," arXiv:2402.17764, Microsoft Research, 2024.

[3] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018.

[4] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," arXiv:1308.3432, 2013.

[5] G. Hinton, "Neural Networks for Machine Learning — Lecture 15," Coursera, 2012 (origin of the Straight-Through Estimator concept).

Appendix A: Software Availability

The complete source code is available at: **github.com/bxf1001g/bitnet-edge**

Licensed under the MIT License. Copyright (c) 2026 Bhanu Chandar — Axonyx Quantum Private Limited.

Appendix B: Zenodo Metadata

Title: bitnet-edge: Ternary-Weight Convolutional Neural Networks for Multiply-Free Inference on Edge Microcontrollers

Authors: Bhanu Chandar (Axonyx Quantum Private Limited)

Description: End-to-end pipeline for training ternary-weight ({-1, 0, +1}) convolutional neural networks in PyTorch and deploying them as multiply-free inference engines on ESP32-S3 microcontrollers. Validated on MNIST (98.79% accuracy, -0.23 pp) and CIFAR-10 (69.02% accuracy, -6.50 pp) with 93.75% weight memory compression and zero multiplications in conv/linear layers. Fully dynamic inference engine supports any input shape without recompilation.

Keywords: ternary quantization; edge inference; multiply-free neural networks; ESP32; CNN; model compression; embedded AI; BitNet; microcontroller; CIFAR-10

License: MIT License

Related identifiers: - GitHub repository: https://github.com/bxf1001g/bitnet-edge (isSupplementedBy) - BitNet b1.58 paper: https://arxiv.org/abs/2402.17764 (isCitedBy)

Resource type: Software / Technical Report

Language: English